GUIDE 22 · ACCESS

Keep connection passwords out of the store

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=22-encrypt-passwords-at-rest. This copy was made on 4 November 2026.

Time: 10 minutes. **You'll end up with:** no readable database password in `queryapigate.db` - either because the store only holds a reference to an environment variable, or because the password is encrypted with a key that lives somewhere else.

By default a connection's password is stored as you typed it. The API never returns it (`GET` shows `********`), but anyone who can read `queryapigate.db` - or a backup of it - can. The server warns about this at startup:

```text
WARNING queryapigate: Connection(s) shop store a literal password in /data/queryapigate.db. Consider a
"${VAR}"
reference to an environment variable instead - it reads the same way but keeps the secret out of the
file, or set
QUERYAPIGATE_SECRET_KEY to encrypt it at rest automatically.
```

There are two ways to fix it. They can be mixed, connection by connection.

Option 1: a `${VAR}` reference (no key to manage)

Store the *name* of an environment variable instead of the password:

```shell
curl -X PATCH http://127.0.0.1:5000/api/v1/connections/warehouse -H 'X-API-Key: demo-key' -H 'Content-
Type: application/json' \
  -d '{"password": "${WAREHOUSE_PASSWORD}"}'
```

and start the server with it set: `WAREHOUSE_PASSWORD=... queryapigate serve`. Verified: the store holds the literal text `${WAREHOUSE_PASSWORD}`, the connection works, and the value is read from the environment each time a connection is opened. Your secret manager (Docker/Kubernetes secrets, systemd credentials, a `.env` file with `chmod 600`) keeps the real value.

Good when you already manage secrets as environment variables. The cost: every new connection needs a variable and a restart.

Option 2: encrypt with `QUERYAPIGATE_SECRET_KEY`

Step 1: Generate a key

```shell
pip install "queryapigate[encryption]"   # the cryptography package; included in [all] and the Docker
image
python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
# xwq0Mh5PZ1o7m-....=
```

Store this key outside `QUERYAPIGATE_HOME` - in your secret manager - and outside any backup of the store. A backup and its key kept together protect nothing.

Step 2: Start the server with it

```shell
QUERYAPIGATE_SECRET_KEY='xwq0Mh5PZ1o7m-....=' QUERYAPIGATE_API_KEY=demo-key queryapigate serve
```

At startup every literal password already in the store is encrypted; from then on, every password saved is encrypted before it's written. Verified on a PostgreSQL connection - the stored record before:

```json
{"host": "localhost", "port": 55433, "database": "postgres", "user": "postgres", "password": "pg-secret-pw"}
```

and after one restart with the key:

```json
{"host": "localhost", "port": 55433, "database": "postgres", "user": "postgres",
 "password":
"enc:gAAAAABqwlPwNP_YGXVRoSW9dmtVHj91VcCJ1yhqFWIHPNtqitMchd5uyQNPISe3g1U5VJCXboOxjq12k7WJY_WsHDx_vJIAjQ==
="}
```

Queries on the connection work as before; the password is decrypted in memory only when a database connection is opened. The API and the audit log still show `********`, and **Settings** shows *Encryption at rest: enabled*. `${VAR}` references are left alone - there's nothing secret in them to encrypt.

A malformed key stops the server at startup rather than running half-protected: *QUERYAPIGATE_SECRET_KEY must be a valid Fernet key - 32 url-safe base64-encoded bytes*.

If the key goes missing

Nothing silently falls back to plaintext. Verified:

- **Started without the key:** a startup warning - *Connection(s) shop, warehouse have an encrypted password but QUERYAPIGATE_SECRET_KEY is not set - they cannot be used until the key that encrypted them is restored* - and each query on them fails with `500`, `"code": "secret_key_required"`.
- **Started with a different key:** `500`, `"code": "password_undecryptable"` - *This connection's password could not be decrypted with QUERYAPIGATE_SECRET_KEY - it was encrypted with a key that is no longer listed. Add that key back (after the first), or enter the password again.*

Both also raise a **connection failing** alert once runs start failing. There is no way to recover an encrypted password without its key - which is the point. Re-enter the password, or restore the key.

Changing the key

`QUERYAPIGATE_SECRET_KEY` can list several keys, comma-separated: the **first** encrypts, **every one** decrypts. So a rotation never takes a connection down (new in 0.22, verified against PostgreSQL connections `warehouse` and `shop`):

1. Generate the new key, and restart with it **first** and the old one after it:

```shell
QUERYAPIGATE_SECRET_KEY=<new-key>,<old-key>
```

   Every connection keeps working - the old key still decrypts them - and the server says what's left to do: *2 password(s) are still encrypted with an old key in QUERYAPIGATE_SECRET_KEY - run `queryapigate rekey` to move them to the first key, then remove the others.* Settings and Alerts say the same.

2. Re-encrypt everything under the new key, on the server (with the same `QUERYAPIGATE_SECRET_KEY`):

```shell
queryapigate rekey --check
# 2 encrypted password(s), 2 key(s) listed: 0 under the first key, 2 under key 2.
# Run `queryapigate rekey` to move them to the first key.
queryapigate rekey
# Re-encrypted 2 password(s) under the first key.
# 2 encrypted password(s), 2 key(s) listed: 2 under the first key, 0 under key 2.
# Nothing is left under the other keys: remove them from QUERYAPIGATE_SECRET_KEY on every instance.
```

   Or, with no shell, as an owner: `POST /api/v1/settings/rekey` (`{"check": true}` to only report). It covers connection and destination passwords, in one transaction - all of them or, if any can't be decrypted by a listed key, none (*No key in QUERYAPIGATE_SECRET_KEY decrypts the password of connection 'shop', … - nothing was changed*). Running it twice is harmless. The audit log records `rekey`, with counts only.

3. Restart with the new key alone, `QUERYAPIGATE_SECRET_KEY=<new-key>`, and delete the old one from your secret manager. Queries work as before; started with the old key alone, they now fail with `password_undecryptable`.

Several instances: do step 1 on all of them before step 2. While they restart one by one, one still on the old key alone can't read a password just saved by one on `new,old` - to avoid even that, roll out `<old-key>,<new-key>` first (still encrypting with the old one), then `<new-key>,<old-key>`.

Backups taken before step 2 need the old key to read their passwords - keep it as long as you keep them.

A key that isn't valid stops the server, and `rekey`, naming its place in the list: *Key 2 of 2 in QUERYAPIGATE_SECRET_KEY must be a valid Fernet key*. The same key twice is refused too.

What this does and doesn't protect

- **Protects:** the store file and its backups, if someone gets one without the key.
- **Doesn't protect:** a running server - it holds the key in memory - or anyone who can read its environment. Database-side protection still matters: give QueryAPIGate a database user with only the rights it needs.
- **Covers connection passwords only.** API keys are never stored at all (only a SHA-256 hash). Other connection fields - host, user, `options` - are stored as given.

Next steps

- Back up and restore the store - and what a backup doesn't contain: this key.
- Deploy with Docker for real - passing the key as a secret.
- Read this project's threat model.